Execution-Aware Limit Order Book Alpha Research

Stage 1: From Predictive Signal to Queue-Aware Production Handoff

Ayan Maiti

September 2026

Abstract

This report documents the methodology implemented in the Jupyter notebook `exploration_stage1_clean`. The project begins with an apparently standard high-frequency alpha problem—predict future executable returns from a five-level LOBSTER limit order book—and evolves as successive diagnostics reveal that statistical predictability is not equivalent to tradable profitability. The research pipeline is therefore changed in response to evidence: first from short-horizon regression to a longer horizon, then from aggressive execution to passive entry, then from optimistic price-touch fills to queue-aware fills, and finally from raw executable-return prediction to a sparse queue-aware trade-state classification problem. The report gives mathematical definitions for the engineered features, execution models, targets, validation rules, and production handoff; records the observed outcomes; explains plausible statistical reasons for those outcomes; and describes limitations and improvements.

Contents

1 Research Objective and Data **2**

2 Core Feature Engineering **2**
2.1 Spread and microprice displacement . . . 2
2.2 Depth imbalance . . . 2
2.3 Price gaps and short-horizon returns . . . 3
2.4 Message-flow features . . . 3

3 Execution Model **3**
3.1 Latency and horizon . . . 3
3.2 Depth-walking VWAP . . . 3
3.3 Executable-return targets . . . 4

4 Stage 1A: Original H=100 Regression **4**
4.1 Observed result . . . 4

5 Methodological Change 1: Horizon Search **4**

6 Methodological Change 2: Cost Ablation **5**

7 Methodological Change 3: Passive Entry **5**

8 Methodological Change 4: Queue-Aware Passive Fills **6**

9 Adverse Selection **6**

10 Methodological Change 5: Queue-Aware Trade-State Target **6**

11 Three-Class Classification and Thresholding **7**
11.1 Validation results 7

12 Untouched Test Evaluation **8**

13 Statistical Interpretation of the Outcome **8**
13.1 Severe class imbalance 8
13.2 Execution costs shift the conditional distribution 8
13.3 Selection on fills creates adverse selection 8
13.4 Threshold overfitting with tiny trade counts 8
13.5 Dependence between adjacent LOB observations 9

14 Production Handoff **9**

15 Limitations **9**

16 Possible Improvements **10**
16.1 Statistical and machine-learning improvements 10
16.2 Microstructure improvements 10
16.3 Mathematical alpha enhancements 10
16.4 Production improvements 10

17 Codeblock-to-Mathematics Map **10**

18 Conclusion **11**

1 Research Objective and Data

The objective is not merely to predict short-horizon price direction. It is to construct an alpha signal that survives realistic execution assumptions. The notebook uses a LOBSTER sample for AAPL on 2012-06-21. The order-book file contains displayed price and size at multiple levels, while the aligned message file contains event time, event type, order identifier, event size, event price, and direction. The implementation reads the first five levels and keeps the message and order-book rows aligned.

At event index t, write the best ask and bid as $A_{t,1}$ and $B_{t,1}$, and their displayed sizes as $Q^a_{t,1}$ and $Q^b_{t,1}$. More generally, $A_{t,\ell}, B_{t,\ell}, Q^a_{t,\ell}, Q^b_{t,\ell}$ denote level $\ell \in \{1, \dots, 5\}$.

The midprice is

$$M_t = \frac{A_{t,1} + B_{t,1}}{2}. \tag{1}$$

The top-of-book microprice used in the notebook is

$$\mu_t = \frac{A_{t,1}Q^b_{t,1} + B_{t,1}Q^a_{t,1}}{Q^b_{t,1} + Q^a_{t,1}}. \tag{2}$$

It tilts the midpoint toward the side with less displayed depth and is intended to encode short-term pressure.

2 Core Feature Engineering

The research frame transforms contemporaneous order-book and message information into predictors X_t that are observable at the signal event.

2.1 Spread and microprice displacement

The spread in basis points is

$$s_t = 10^4 \frac{A_{t,1} - B_{t,1}}{M_t}, \tag{3}$$

and the microprice deviation is

$$d^\mu_t = 10^4 \frac{\mu_t - M_t}{M_t}. \tag{4}$$

2.2 Depth imbalance

For depth $d \in \{1, 3, 5\}$ define cumulative bid and ask depth

$$Q^b_t(d) = \sum_{\ell=1}^{d} Q^b_{t,\ell}, \qquad Q^a_t(d) = \sum_{\ell=1}^{d} Q^a_{t,\ell}. \tag{5}$$

The corresponding imbalance is

$$I_t(d) = \frac{Q^b_t(d) - Q^a_t(d)}{Q^b_t(d) + Q^a_t(d)}. \tag{6}$$

The notebook also uses log-depth transforms $\log(1 + Q^b_t(d))$ and $\log(1 + Q^a_t(d))$.

A near-touch-weighted imbalance gives weight $w_\ell = 1/\ell$ to level ℓ:

$$I^w_t = \frac{\sum_{\ell=1}^{5} w_\ell Q^b_{t,\ell} - \sum_{\ell=1}^{5} w_\ell Q^a_{t,\ell}}{\sum_{\ell=1}^{5} w_\ell Q^b_{t,\ell} + \sum_{\ell=1}^{5} w_\ell Q^a_{t,\ell}}. \tag{7}$$

2.3 Price gaps and short-horizon returns

For each level,

$$g^a_{t,\ell} = 10^4 \frac{A_{t,\ell} - M_t}{M_t}, \qquad g^b_{t,\ell} = 10^4 \frac{M_t - B_{t,\ell}}{M_t}. \tag{8}$$

Log-mid returns are computed at event lags $k \in \{1, 5, 20\}$:

$$r^{(k)}_t = 10^4 \left(\log M_t - \log M_{t-k}\right). \tag{9}$$

Rolling realized volatility over a window w is the sample standard deviation of one-event log-mid returns.

2.4 Message-flow features

For event windows $w \in \{20, 100, 500\}$, the notebook forms signed trade flow, submission flow, cancellation pressure, net message pressure, execution fraction, cancellation fraction, and event rate. A generic normalized signed flow has the form

$$F_{t,w} = \frac{\sum_{j=t-w+1}^{t} f^{\text{signed}}_j}{\sum_{j=t-w+1}^{t} |f_j|}, \tag{10}$$

with zero assigned to a complete window containing no corresponding flow.

The notebook also computes a best-level order-flow-imbalance increment and rolling normalized OFI. In stylized form,

$$\text{OFI}_t = \mathbf{1}_{\{B_t \geq B_{t-1}\}} Q^b_t - \mathbf{1}_{\{B_t \leq B_{t-1}\}} Q^b_{t-1} - \mathbf{1}_{\{A_t \leq A_{t-1}\}} Q^a_t + \mathbf{1}_{\{A_t \geq A_{t-1}\}} Q^a_{t-1}, \tag{11}$$

which is accumulated over the same rolling windows and normalized by average top-level depth.

3 Execution Model

3.1 Latency and horizon

A signal is generated at event t. The model assumes an entry latency $L = 10$ events. For a horizon H, the entry snapshot is $t + L$ and the exit snapshot is $t + L + H$.

For a fixed notional $N = \$10{,}000$, the order size is approximately

$$q_t = \left\lfloor \frac{N}{M_{t+L}} \right\rfloor. \tag{12}$$

3.2 Depth-walking VWAP

For a market buy of q shares, displayed asks are consumed from the best level outward. If q_ℓ shares are taken at level ℓ,

$$q_\ell = \min\left(Q^a_{t,\ell},\ q - \sum_{j<\ell} q_j \right), \tag{13}$$

and the execution VWAP is

$$P^{\text{buy}}_t(q) = \frac{\sum_{\ell=1}^{5} q_\ell A_{t,\ell}}{\sum_{\ell=1}^{5} q_\ell}. \tag{14}$$

The sell VWAP is defined analogously using bids. Orders that cannot be filled within the five displayed levels are rejected. The original execution model additionally applies one tick of slippage per side.

3.3 Executable-return targets

With aggressive entry and aggressive exit, the long target is

$$R_t^L = 10^4 \frac{P_{t+L+H}^{\text{sell}} - P_{t+L}^{\text{buy}}}{P_{t+L}^{\text{buy}}} - C, \tag{15}$$

and the short target is

$$R_t^S = 10^4 \frac{P_{t+L}^{\text{sell}} - P_{t+L+H}^{\text{buy}}}{P_{t+L}^{\text{sell}}} - C, \tag{16}$$

where C is round-trip commission in basis points. In the notebook commissions are set to zero, so the measured drag comes from spread, depth and the configured extra slippage.

4 Stage 1A: Original H=100 Regression

Three regressors are compared separately for long and short executable returns: Ridge regression, histogram gradient boosting, and random forest. The chronological data are split into 60% train, 20% validation, and 20% test with a purge equal to latency plus horizon to reduce leakage from overlapping future targets.

The trading rule is

$$a_t = \begin{cases} +1, & \widehat{R}_t^L \geq \widehat{R}_t^S \text{ and } \widehat{R}_t^L > \delta, \\ -1, & \widehat{R}_t^S > \widehat{R}_t^L \text{ and } \widehat{R}_t^S > \delta, \\ 0, & \text{otherwise}, \end{cases} \tag{17}$$

where δ is selected on validation economics.

4.1 Observed result

At $H = 100$, the selected Ridge model produced no validation trades. Mean validation executable returns were -2.7901 bps long and -2.7205 bps short. Only 2.14% of long rows and 2.49% of short rows were profitable, while every predicted edge was negative. Nevertheless, directional accuracies were 97.37% and 96.72%.

This apparent contradiction is statistical, not mysterious. A classifier or sign metric can be highly accurate when the target is overwhelmingly negative. Here the high directional accuracy mostly reflects correct prediction of the dominant negative class; it does not imply exploitable positive edge. The information coefficients of roughly 0.40–0.42 show ranking information, but the entire conditional distribution is shifted below zero by execution costs.

The untouched test set generated only one trade, with net return -1.2069 bps and PnL $-\$1.21$.

5 Methodological Change 1: Horizon Search

Because $H = 100$ produced almost no positive executable opportunities, the notebook evaluates $H \in \{10, 25, 50, 100, 250, 500, 1000\}$.

H	$\mathbb{E}\lvert\Delta M\rvert$ bps	Long prof. %	Short prof. %	Oracle best bps
10	0.221	0.005	0.020	-2.536
25	0.407	0.031	0.120	-2.349
50	0.629	0.287	0.534	-2.127
100	0.973	2.144	2.491	-1.783
250	1.649	10.107	10.878	-1.099
500	2.358	16.772	20.625	-0.389
1000	3.185	23.656	26.589	0.446

Table 1: Validation horizon diagnostic from the notebook. "Oracle best" chooses the better realized side row-by-row and is therefore not a tradable forecast; it is an upper-bound diagnostic.

The first horizon with positive oracle mean best executable return is $H = 1000$. This motivates moving subsequent execution research to $H = 1000$. The change is evidence-driven: price movement grows with horizon while the approximately fixed spread-crossing cost becomes less dominant relative to the signal scale.

6 Methodological Change 2: Cost Ablation

The notebook decomposes execution drag using four models.

Execution model	Mean long	Mean short	Oracle best
Midpoint / zero cost	-0.317	0.317	3.185
Best bid-ask / spread only	-2.641	-2.007	0.861
Depth VWAP / no extra slippage	-2.724	-2.089	0.790
Depth VWAP / +1 tick	-3.068	-2.433	0.446

Table 2: Cost ablation at $H = 1000$, in basis points.

The largest drop is caused by crossing the spread: the oracle mean falls from 3.185 bps at midpoint to 0.861 bps at best-touch execution. Additional depth costs reduce it only to 0.790 bps, and the extra one-tick slippage reduces it to 0.446 bps. Thus the dominant structural problem is taker/taker spread crossing, not displayed-depth impact.

7 Methodological Change 3: Passive Entry

A passive buy is posted at the best bid at $t + L$, and a passive sell at the best ask. The first diagnostic is optimistic: a fill is recorded if a visible execution occurs at the exact quoted price within 100 events. Under this price-touch rule, validation fill rates are 32.98% for buys and 33.06% for sells, with median fill times of 19 and 20 events.

Conditional on these optimistic fills, passive-entry/aggressive-exit mean returns improve relative to taker/taker execution but remain negative:

$$\mathbb{E}[R^{L,\text{passive}} \mid \text{fill}] = -1.6767\,\text{bps}, \qquad \mathbb{E}[R^{S,\text{passive}} \mid \text{fill}] = -1.1879\,\text{bps}. \tag{18}$$

Hence saving the entry-side spread is helpful but insufficient.

8 Methodological Change 4: Queue-Aware Passive Fills

A price touch does not imply that a newly submitted order is filled because displayed volume may already be ahead in the queue. The notebook therefore introduces a conservative queue approximation. At arrival, the displayed best-quote size is treated as queue ahead, Q_t^{ahead}, and the new order is assumed to join at the back of that queue. A full fill occurs only when cumulative visible executions at exactly the quoted price satisfy

$$V_t^{\text{exec}}(k) \geq Q_t^{\text{ahead}} + q_t. \tag{19}$$

Cancellations are ignored, which makes the rule conservative.

This changes the empirical fill rate dramatically:

$$\mathbb{P}(\text{buy fill}) = 6.61\%, \qquad \text{median fill time} = 36, \tag{20}$$

$$\mathbb{P}(\text{sell fill}) = 5.87\%, \qquad \text{median fill time} = 38. \tag{21}$$

The number of validation fills falls to 5,203 buys and 4,623 sells. Queue-aware passive-entry/aggressive-exit mean returns remain negative at -1.5491 bps long and -1.5560 bps short.

9 Adverse Selection

To diagnose whether fills are systematically toxic, the notebook measures signed midpoint movement after a queue-aware fill. For a buy, positive means the midpoint subsequently rises; for a sell, the sign is reversed so positive again means favorable movement.

Post-fill events	Buy mean bps	Sell mean bps	Interpretation
10	-0.182	-0.166	adverse
25	-0.243	-0.265	adverse
50	-0.436	-0.406	adverse
100	-0.532	-0.633	adverse
250	-0.623	-0.728	adverse

Table 3: Signed post-fill midpoint movement. Negative values mean the market moves against the passive position.

The result is persistent adverse selection. The order tends to fill precisely when aggressive flow is strong enough to consume the queue, so the fill event is endogenous: fill probability and unfavorable subsequent movement are linked.

A fill-speed diagnostic does not support the simple claim that faster fills are always more toxic. Buy fills in 1–10 events have mean eventual return -0.80 bps, less negative than slower buy fills; sell results are non-monotonic. Therefore the notebook does not impose an arbitrary "reject fast fills" rule.

10 Methodological Change 5: Queue-Aware Trade-State Target

The earlier three-class target used aggressive-entry/aggressive-exit returns. After the execution diagnostics, the target is redefined to represent the actual intended mechanism: queue-aware passive entry and aggressive exit.

Define the realized passive returns for every row as

$$R_t^{L,p} = 10^4 \frac{P_{t+L+H}^{\text{sell}} - B_{t+L,1}}{B_{t+L,1}}, \qquad R_t^{S,p} = 10^4 \frac{A_{t+L,1} - P_{t+L+H}^{\text{buy}}}{A_{t+L,1}}. \tag{22}$$

Let $F_t^B, F_t^S \in \{0,1\}$ denote queue-aware buy and sell fills. The trade-state label is

$$Y_t = \begin{cases} +1, & F_t^B = 1,\ R_t^{L,p} > 0,\ R_t^{L,p} \geq R_t^{S,p}, \\ -1, & F_t^S = 1,\ R_t^{S,p} > 0,\ R_t^{S,p} > R_t^{L,p}, \\ 0, & \text{otherwise.} \end{cases} \tag{23}$$

On validation the new target is extremely sparse:

$$\mathbb{P}(Y = -1) = 2.21\%, \tag{24}$$

$$\mathbb{P}(Y = 0) = 95.47\%, \tag{25}$$

$$\mathbb{P}(Y = +1) = 2.32\%. \tag{26}$$

The profitable LONG class contains 1,826 rows with mean realized return 2.3985 bps; the SHORT class contains 1,742 rows with mean 2.1375 bps.

This target construction is deliberately ex post: fill and future return define the label but are not included in the feature set. The prediction problem is therefore to identify, from X_t alone, rare states in which a future queue-aware trade will both fill and be profitable.

11 Three-Class Classification and Thresholding

The notebook compares Logistic Regression, Histogram Gradient Boosting, and Random Forest classifiers for $Y_t \in \{-1, 0, +1\}$. Let

$$(p_t^-, p_t^0, p_t^+) = f(X_t) \tag{27}$$

be predicted class probabilities. At confidence threshold δ, a long order is submitted when

$$p_t^+ \geq \delta, \quad p_t^+ > p_t^0, \quad p_t^+ \geq p_t^-, \tag{28}$$

and a short order is submitted symmetrically. Submission still does not imply a trade: the queue-aware fill condition must be satisfied. Filled positions are kept non-overlapping until the $H = 1000$ exit; unfilled orders are cancelled after 100 events.

11.1 Validation results

Model	Accuracy	Balanced acc.	Selected δ	Validation PnL
HistGradientBoosting	0.9547	0.3342	0.30	+\$5.50 (1 trade)
Logistic	0.3517	0.4599	0.70	−\$5.15 (9 trades)
Random Forest	0.9285	0.3554	0.50	−\$5.84 (7 trades)

Table 4: Queue-aware three-class validation results.

The raw 95.47% accuracy of Histogram Gradient Boosting is not evidence of a strong trading classifier by itself: the validation NO-TRADE base rate is also 95.47%. Balanced accuracy near 1/3 means the model barely exceeds chance across the three classes. Logistic Regression has much lower ordinary accuracy but higher balanced accuracy because it predicts minority classes more often.

The confidence diagnostic is also informative. For Logistic Regression, high thresholds sharply reduce sample size. At $\delta = 0.75$, only 18 filled signals remain in the non-overlap-free diagnostic and mean filled return is approximately 0.024 bps; at $\delta = 0.80$, only six fills remain with mean 1.833 bps. This suggests possible tail ranking, but the sample is too small to establish a stable edge. The non-overlapping threshold sweep similarly shows positive outcomes only at very high confidence with one or a few trades.

12 Untouched Test Evaluation

The validation-selected model is Histogram Gradient Boosting with $\delta = 0.30$. It is frozen before the test evaluation. On the chronological test set:

$$\text{accuracy} = 0.9091, \tag{29}$$

$$\text{balanced accuracy} = 0.3342. \tag{30}$$

Exactly one queue-aware filled trade is generated. It is a long with predicted probability 0.5899, fills after 37 events, and realizes -1.2121 bps, i.e. approximately $-\$1.21$ on the $\$10,000$ notional.

The correct conclusion is therefore not that Stage 1 found a profitable HFT strategy. Stage 1 found measurable predictive structure and identified why that structure did not translate into robust PnL under increasingly realistic execution. The final test evidence is far too sparse to support a profitability claim.

13 Statistical Interpretation of the Outcome

Several statistical mechanisms can explain the observed pattern.

13.1 Severe class imbalance

The final target has 95.47% NO TRADE labels. Ordinary accuracy is dominated by the majority class. A trivial classifier that always predicts NO TRADE already obtains approximately 95.47% validation accuracy. For this reason, balanced accuracy, minority-class precision/recall, PR-AUC, calibrated probabilities, and realized PnL are more relevant than raw accuracy.

13.2 Execution costs shift the conditional distribution

The original regressors have nontrivial information coefficients, yet predicted executable edges are negative. This is consistent with a model that ranks outcomes reasonably while the full distribution of net returns is shifted below zero by the spread. Ranking skill is necessary but not sufficient when the mean cost is larger than the conditional alpha.

13.3 Selection on fills creates adverse selection

A passive fill is not missing at random. The event $F_t = 1$ occurs more often when aggressive flow reaches the resting quote. Conditioning on $F_t = 1$ therefore changes the return distribution:

$$\mathbb{E}[R_{t+h} \mid F_t = 1, X_t] \neq \mathbb{E}[R_{t+h} \mid X_t]. \tag{31}$$

The negative post-fill drift in the notebook is direct empirical evidence of this selection effect.

13.4 Threshold overfitting with tiny trade counts

Validation thresholds that look profitable often correspond to one or a handful of trades. Maximizing validation PnL over many thresholds induces selection bias. The maximum of several noisy estimates tends to be upward biased. This is especially severe when effective sample size is small because non-overlap and queue fills remove most candidate events.

13.5 Dependence between adjacent LOB observations

Rows are event-by-event observations from one trading day and are strongly dependent. Even with purged chronological splits, effective sample size is far smaller than the row count because order-book states are serially correlated. This makes naive standard errors or row-count intuition overly optimistic.

14 Production Handoff

The notebook does include a production-oriented handoff. After model and threshold selection, it saves a serialized bundle containing

- the frozen classifier,
- model name,
- feature-name schema,
- validation-selected probability threshold,
- class ordering,
- configuration parameters, and
- passive fill horizon.

The artifact is written as `queue_aware_alpha_model.joblib`. This is a meaningful production step because inference should not redo model selection or threshold optimization. A production signal service should load the frozen bundle, validate feature schema, compute probabilities, and emit LONG, SHORT, or NO TRADE without access to future fill or return information.

The project intentionally stops before live order routing. A robust next production layer would add deterministic feature computation from streaming messages, schema/version checks, latency monitoring, model health metrics, shadow replay, structured logs, and risk controls.

15 Limitations

1. **Single-day sample.** The notebook uses a one-day AAPL LOBSTER sample. Intraday behavior, spread, volatility, event intensity, and queue dynamics may not generalize across days or securities.
2. **Approximate queue model.** The queue-aware rule treats displayed size at arrival as fully ahead and counts only visible executions at the exact price. It ignores cancellations ahead, new orders behind, hidden liquidity, order priority details, and exact order-ID queue reconstruction.
3. **No exchange fees/rebates.** Commissions are set to zero. Maker rebates and taker fees can materially change HFT economics.
4. **Simplified latency.** Latency is represented as a fixed number of events rather than wall-clock network, exchange, and processing latency.
5. **Five displayed levels.** Market orders that exceed the observed depth are rejected rather than modeled using deeper liquidity.
6. **Fixed notional.** Position size is fixed at $\$10,000$; there is no volatility scaling, inventory constraint, or market-impact optimization.
7. **Sparse final test trades.** One test trade is not enough to estimate Sharpe ratio, tail risk, drawdown, or stable expected PnL.
8. **Threshold selection noise.** Validation threshold optimization with very few fills is prone to overfitting.
9. **No probability calibration diagnostics.** The notebook uses class probabilities operationally but does not yet assess calibration curves, Brier score, or reliability by confidence bucket.
10. **No multi-day walk-forward validation.** A production research process should validate chronologically across many days and market regimes.

16 Possible Improvements

16.1 Statistical and machine-learning improvements

Use multi-day walk-forward splits with day-level grouping and purging, report PR-AUC for the rare profitable classes, calibrate probabilities, and quantify uncertainty through block bootstrap or day-level resampling. Cost-sensitive classification or focal loss can be compared with the current models, but model complexity should only be accepted when out-of-sample execution PnL improves.

16.2 Microstructure improvements

Reconstruct queue position using order IDs when possible; explicitly model cancellations ahead of the order; distinguish hidden executions; model maker rebates and exchange fees; and test passive exit as well as passive entry. Fill probability and adverse selection can be modeled jointly rather than treating fill as a deterministic ex post filter.

16.3 Mathematical alpha enhancements

The next research stage can add stochastic-process features without discarding the current pipeline. Examples include a fair-price or martingale residual

$$\alpha_t = F_t - M_t, \tag{32}$$

a mean-reverting latent alpha model

$$dZ_t = -\kappa Z_t\,dt + \beta^\top X_t\,dt + \sigma\,dW_t, \tag{33}$$

and event-intensity models for bid/ask depletion. A Hawkes-process formulation

$$\lambda_i(t) = \mu_i + \sum_j \int_0^t \phi_{ij}(t-s)\,dN_j(s) \tag{34}$$

is especially natural for clustered submissions, cancellations, and executions. These models should be judged by incremental out-of-sample queue-aware PnL rather than mathematical sophistication alone.

16.4 Production improvements

Refactor notebook logic into tested modules; create a deterministic feature pipeline shared by training and inference; freeze data and model versions; add model-card metadata; perform shadow replay on unseen days; track latency and feature drift; and expose an inference interface that returns probabilities and action together with explicit abstention.

17 Codeblock-to-Mathematics Map

Block	Notebook purpose	Mathematical interpretation
1	Imports	Numerical, statistical, serialization and plotting dependencies; no model assumption.
2	Configuration	Defines L, H, N, depth level, slippage, fill horizon, random seed, and message-flow windows.
3	LOBSTER schema helper	Maps raw columns to $(A_{t,\ell}, Q^a_{t,\ell}, B_{t,\ell}, Q^b_{t,\ell})$.

Block	Notebook purpose	Mathematical interpretation
4	Aligned data loader	Ensures message event e_t and order-book state $\mathcal{B}_t$ share the same event index.
5	`depth_vwap`	Computes $P_t^{buy/sell}(q)$ by sequentially consuming displayed levels.
6	Research frame	Constructs X_t, executable returns, passive fill diagnostics, queue variables, and cost-ablation outputs.
7	Regression candidates	Estimates $\mathbb{E}[R_t^L \mid X_t]$ and $\mathbb{E}[R_t^S \mid X_t]$ using linear and nonlinear models.
8	Chronological split	Produces time-ordered train/validation/test sets with purge $L + H$.
9	Trading simulator	Implements $a_t \in \{-1, 0, +1\}$ with non-overlap and realized executable PnL.
10	Threshold selection	Chooses δ on validation economics rather than test data.
11–12	Output helpers / pipeline	Serializes evaluation, model, trades, feature importance and plots.
13–16	Paths, run, inspection	Reproducible project-root discovery, artifact directory, baseline training and result inspection.
17	Horizon diagnostic	Compares H by $\mathbb{E}\lvert\Delta M\rvert$, profitable-row fractions and oracle executable upper bound.
18	$H = 1000$ state diagnostic	Defines an initial ex-post long/short/no-trade partition to assess opportunity density.
19	Cost ablation	Separates midpoint alpha, spread, depth and added-slippage components.
20	Optimistic passive fill	Tests whether visible execution occurs at the submitted best quote within 100 events.
21	Queue-aware fill	Requires cumulative execution $\geq Q^{ahead} + q$; evaluates fill-conditioned PnL and post-fill drift.
22	Queue-aware target	Defines $Y_t \in \{-1, 0, +1\}$ from fill status and positive passive-entry executable return.
23–27	Classification / threshold / confidence	Estimates $\mathbb{P}(Y_t = c \mid X_t)$, applies confidence threshold, queue-aware fill and non-overlap, and inspects ranking by confidence.
28	Freeze model	Ends validation selection and fixes model plus threshold.
29	Untouched test	Applies the frozen policy exactly once to chronological test data.
30	Production handoff	Serializes $(f, \delta, \text{features}, \text{classes}, \text{config})$ into a deployable bundle.

18 Conclusion

Stage 1 is best viewed as an execution-research result rather than a profitable-strategy result. The notebook demonstrates that seemingly strong predictive metrics can collapse once spread, queue position, fills, and adverse selection are incorporated. The work progressively replaces optimistic assumptions with more realistic ones, and every major methodological change is motivated by an observed failure mode. The final out-of-sample result—one filled trade with negative PnL—does not establish an edge, but the pipeline provides a credible foundation for Stage 2: multi-day validation, stronger queue modeling, stochastic-process alpha features, and production-grade shadow replay.